SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA3715**

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I B.Hari Sai/192311098 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
B.Hari Sai

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Testing as an Engineering Activity

Testing should follow a planned, measurable process instead of random checks. The team should study requirements, identify risks, design test cases with expected results, prepare test data, execute tests, record defects, and retest fixes. Techniques such as equivalence partitioning and boundary value analysis improve coverage. For example, a registration form should be tested with valid, invalid, empty, and boundary-length inputs. This systematic approach makes results repeatable and detects defects earlier.

Q2. Defined Testing Process in a Banking System

A defined process includes test planning, requirement review, test design, execution, defect reporting, retesting, regression testing, and sign-off. Banking tests should cover deposits, withdrawals, transfers, balances, security, and recovery. For example, an ATM withdrawal test must verify that the account is debited only once and cash is dispensed correctly. Standard procedures improve traceability, reliability, compliance, and customer confidence.

Q3. Testing as a Structured SDLC Process

Testing should be planned throughout the SDLC: review requirements, prepare test plans, design cases, test individual units, test integrations, test the complete system, conduct acceptance testing, and perform regression testing after changes. In an e-commerce system, unit tests check price calculations, integration tests check payment and inventory, and system tests check the full order journey. Following this process finds defects at the appropriate stage and reduces rework.

Q4. Error, Defect, and Failure

An error is a human mistake, a defect (bug) is a flaw in a requirement, design, or code caused by that mistake, and a failure is the observable incorrect behavior when the defect executes. For example, a developer mistakenly uses the wrong comparison operator (error), the incorrect condition remains in the code (defect), and a valid login is rejected (failure). These terms describe related stages of a problem.

Q5. Early Testing and the Cost of Defects

Apply the principle that early testing saves time and cost. Review requirements and designs before coding, then test each component as it is built. A requirement ambiguity found during review is usually cheaper to correct than a defect discovered after deployment. For example, clarifying password rules before implementation prevents code changes, retesting, and customer support costs later.

Q6. Tester Involvement During Requirements

Testers should review healthcare requirements for ambiguity, completeness, consistency, testability, privacy, and safety. They can ask how patient records are updated, who may access them, and what happens when data is missing. Clear acceptance criteria and requirement-based test cases help developers build the correct system and prevent costly defects before coding begins.

Q7. Ambiguous Requirements in a Payroll System

Defects can originate when requirements are incomplete, ambiguous, contradictory, or misunderstood. If overtime rules do not specify rounding or eligibility, developers may implement different interpretations and calculate salaries incorrectly. The team should clarify rules with stakeholders, document examples and acceptance criteria, and trace each requirement to test cases. Clear requirements reduce rework and payroll errors.

Q8. Boundary Value Analysis

A crash at input limits is commonly a boundary-related defect caused by insufficient boundary testing. Boundary value analysis tests values at, just below, and just above limits. If an age field accepts 18–60, test 17, 18, 19, 59, 60, and 61, along with empty or malformed input where relevant. These tests expose off-by-one errors and improper limit handling.

Q9. Using a Defect Repository

A defect repository should record each defect's description, steps to reproduce, severity, priority, affected module, root cause, status, and resolution. Testers can review past defects to create regression tests and target error-prone areas. For example, if date validation repeatedly failed, future releases should include date, leap-year, and invalid-format tests. Reusing this knowledge prevents recurrence and improves coverage.

Q10. Functional Defect in an ATM

An ATM withdrawal failure caused by incorrect transaction logic is a functional defect because the system does not meet the specified behavior. Tests should verify valid and invalid PINs, sufficient and insufficient funds, withdrawal limits, account balance updates, cash dispensing, and network interruptions. Transaction tests must confirm that a failed withdrawal does not wrongly debit the account. Clear requirements and integration testing could have detected the defect.

Q11. Defect Clustering

Defect clustering means that a large proportion of defects are often concentrated in a small number of modules. The team should analyze defect history, code complexity, change frequency, and business criticality to identify these modules. More reviews and deeper testing should be assigned to high-risk areas, while other modules receive appropriate baseline coverage. This focuses effort where it is most likely to find defects.

Q12. Performance Defect

Slow response under heavy load is a non-functional performance defect. The team should conduct load, stress, and endurance tests, measuring response time, throughput, resource use, and error rate. For example, simulate expected peak shopping traffic and then increase it to find capacity limits. Classification helps route the issue to performance specialists and guides investigation of database queries, server resources, or network bottlenecks.

Q13. Historical Data for Regression Testing

Historical defect data identifies previously failing features, modules, and scenarios. Convert resolved defects into regression test cases and run them after code changes, especially in high-risk areas. For example, a past payment duplication defect should be retested after changes to checkout or transaction handling. This preserves coverage, detects regressions early, and avoids spending time rediscovering known problem areas.

Q14. Tester Collaboration During Integration

Testers should communicate expected behavior, interfaces, test data, and defects clearly to developers and analysts. They should attend design and integration discussions, review API contracts, and report reproducible steps with logs or screenshots. Early collaboration can reveal mismatched data formats or assumptions before components are combined, reducing integration failures and shortening resolution time.

Q15. Defect Prevention

To prevent recurring defects, perform root-cause analysis, review code and requirements, establish coding and design standards, use static analysis, automate regression tests, and share lessons learned. If several releases contain the same input-validation bug, identify why existing reviews and tests missed it, then add a standard validation checklist and automated cases. Prevention reduces repeated rework and improves long-term product quality.

Q16. Standardized Testing for a Mobile App

The team should define a test strategy, common templates, risk-based priorities, entry and exit criteria, and a repeatable defect-reporting process. Test on supported devices, operating-system versions, screen sizes, network conditions, and interrupted sessions. Automate stable regression tests and manually explore new risks. Standardization makes results consistent and improves app reliability across devices.

Q17. Testing vs. Debugging

Testing executes software to reveal failures and compare actual results with expected results. Debugging begins after a problem is identified and locates, analyzes, and fixes its cause. For example, a test reveals that a transfer debits an account twice; the developer then traces the transaction logic, fixes the code, and reruns tests. Testing detects problems, while debugging diagnoses and corrects them.

Q18. Cross-Browser Testing

Prepare a browser and operating-system matrix based on user needs, then test core workflows on supported browsers and versions. Check layout, forms, JavaScript behavior, accessibility, downloads, and payment flows. Use real browsers or reliable automation, and prioritize popular browsers and critical functions. For example, verify that checkout totals and validation behave consistently in Chrome, Firefox, Edge, and Safari.

Q19. Validation Gaps in a Logistics Application

Inconsistent outputs may result from missing input validation, incomplete test cases, incorrect business rules, or untested integrations. Validate required fields, data types, ranges, formats, and relationships, then test valid, invalid, missing, and boundary values. Check data at both user-interface and service layers. These practices prevent bad data from entering the system and improve reliability of tracking and delivery calculations.

Q20. Importance of Systematic Test Design

Random testing alone does not ensure coverage. Start from requirements and risks, define preconditions and expected results, and design cases using equivalence partitioning, boundary value analysis, decision tables, and state transitions where suitable. Assign each case an identifier and trace it to a requirement. Systematic design gives repeatable coverage, makes gaps visible, and improves defect detection.

Q21. Data Consistency During Integration

Data inconsistency can arise from mismatched field definitions, incorrect mappings, transaction failures, timing issues, or different assumptions between modules. Test interfaces, data transformations, transaction boundaries, error handling, and concurrent updates. For example, verify that a patient update appears consistently in registration, billing, and laboratory modules. Integration tests and reconciliation checks help preserve accurate, consistent records.

Q22. Tester Roles Across the SDLC

During requirements, testers review clarity and testability; during design, they identify risks and plan coverage; during development, they prepare cases and test units or builds; during integration and system testing, they verify interfaces and end-to-end workflows; and during release, they support acceptance and regression testing. Early involvement catches misunderstandings sooner and reduces late rework.

Q23. Coverage for Peak Usage

Peak-time failures indicate that important load or usage scenarios may be missing. Derive tests from user journeys and risks, include normal, peak, stress, recovery, and concurrent-user scenarios, and measure response time, throughput, and errors. For example, simulate many users checking out simultaneously. Traceability and coverage reviews help reveal untested requirements and reduce production failures.

Q24. Defect Documentation and Tracking

Use a defect repository with a unique ID, title, environment, steps to reproduce, expected and actual results, severity, priority, evidence, owner, status, and resolution. Apply a consistent workflow from new to assigned, fixed, retested, and closed. Good documentation prevents duplicate reports, supports prioritization and trend analysis, and lets testers verify fixes reliably.

Q25. Improving Banking Test Practices

Use risk-based planning and requirements traceability, then perform unit, integration, system, security, performance, and regression testing. Test deposits, withdrawals, transfers, limits, concurrent transactions, timeouts, and recovery. Automate repeatable critical checks and require independent review of high-risk results. This structured approach detects transaction and data-integrity defects and increases banking-system reliability.

Q26. Testing Edge Cases

Testing only normal inputs misses unusual but valid or invalid conditions. Add edge cases such as empty fields, maximum lengths, zero, negative values, duplicate requests, expired sessions, network loss, and values just outside permitted ranges. Boundary value analysis checks values at and around limits. These tests expose hidden assumptions and improve robustness.

Q27. Process-Based Testing Across Modules

Adopt a common testing process with standard plans, case templates, severity definitions, review practices, environments, and reporting. Map requirements to tests and define entry and exit criteria for each phase. Use peer reviews and shared metrics to compare module coverage and defect trends. Standardization reduces inconsistency and ensures every module receives an appropriate level of testing.

Q28. Input Validation for Unexpected Inputs

Test required fields, data types, formats, length limits, ranges, special characters, malformed requests, and unexpected or malicious values. Reject invalid input safely, provide clear messages, and validate on the server as well as the client. For example, a numeric quantity field should reject text and negative values. Strong validation prevents crashes, incorrect processing, and security problems.

Q29. Reducing Defect Leakage to Production

Defect leakage often results from incomplete requirements coverage, weak test data, insufficient integration or regression tests, unrealistic environments, and rushed releases. Analyze production incidents, add missing cases to regression suites, strengthen risk-based and exploratory testing, and use release exit criteria. Monitor production safely and feed findings back into testing. These steps reduce repeated escapes and improve user trust.

Q30. Early Defect Detection Strategy

Review requirements and design early, involve testers throughout the SDLC, use risk-based test planning, automate unit and regression tests, and track requirement coverage. Prioritize critical and frequently changed areas, and report defects with clear evidence. Finding a requirement or coding defect before release usually costs less than correcting it in production, where fixes may require emergency work, data repair, and customer support.